

Apache Spark Architecture

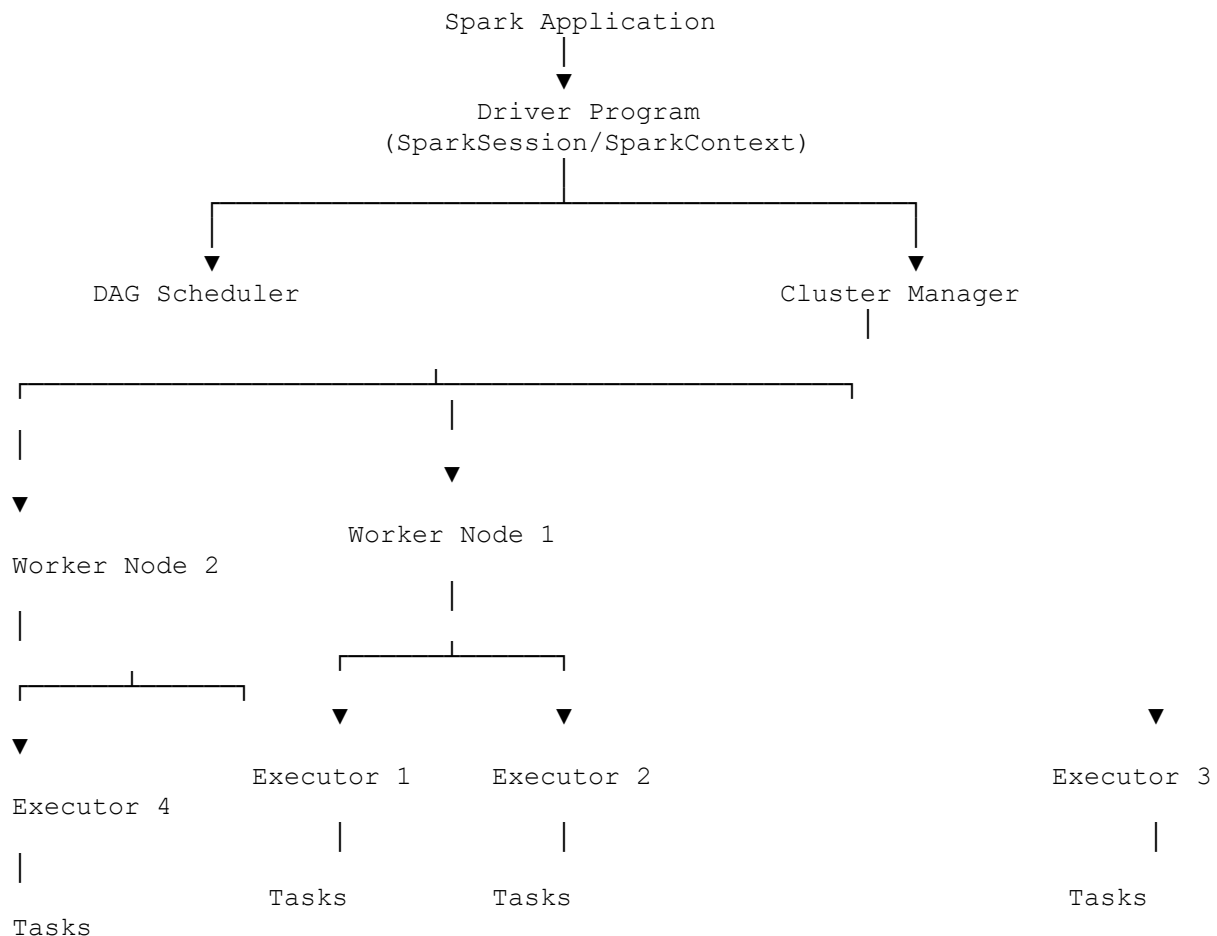
Apache Spark follows a **master-worker architecture** for distributed data processing.

It consists of five main components:

1. Driver
2. Cluster Manager
3. Worker Nodes
4. Executors
5. Tasks

Together, these components allow Spark to process massive datasets in parallel across multiple machines.

Apache Spark Architecture Diagram



1. Driver Program

What is the Driver?

The **Driver** is the **brain** of a Spark application.

It is responsible for:

- Creating the `SparkSession` (and internally the `SparkContext`)
- Reading your Spark code
- Building the execution plan
- Optimizing the plan (using Catalyst for DataFrame/SQL operations)
- Creating the DAG (Directed Acyclic Graph)
- Dividing the DAG into stages
- Splitting each stage into tasks
- Sending tasks to executors
- Monitoring execution
- Collecting results

The Driver runs only **one instance** per Spark application.

2. Cluster Manager

What is the Cluster Manager?

The **Cluster Manager** allocates resources for Spark applications.

It decides:

- How many executors to start
- Where executors will run
- CPU allocation
- Memory allocation

Spark supports multiple cluster managers:

- Standalone Cluster Manager
- Apache YARN
- Kubernetes

- Apache Mesos

The Cluster Manager does **not** execute tasks; it only manages cluster resources.

3. Worker Nodes

What are Worker Nodes?

Worker nodes are the machines that provide the computing resources for Spark jobs.

Each worker node can run one or more executors depending on the available CPU cores and memory.

Responsibilities:

- Host executors
 - Provide CPU and memory resources
 - Execute assigned tasks
-

4. Executors

What are Executors?

Executors are JVM processes launched on worker nodes for a Spark application.

Responsibilities:

- Execute tasks assigned by the Driver
- Read data from storage (HDFS, S3, local files, etc.)
- Perform transformations and actions
- Cache data in memory
- Write results
- Report task status back to the Driver

Executors exist only for the lifetime of the Spark application.

5. Tasks

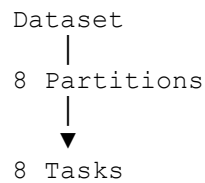
What is a Task?

A **Task** is the smallest unit of work in Spark.

Each task processes **one data partition**.

For example:

If a DataFrame has **8 partitions**, Spark creates **8 tasks** for a stage.



Tasks run in parallel on available executors.

How Spark Executes a Job

Suppose you run:

```
df = spark.read.csv("employees.csv", header=True)

df.filter(df.salary > 5000) \
  .groupBy("department") \
  .count() \
  .show()
```

Step 1: SparkSession

The application starts by creating a `SparkSession`.

Step 2: Driver

The Driver:

- Reads your code
- Builds a logical plan

- Optimizes it
 - Creates a physical plan
 - Builds a DAG of execution
-

Step 3: DAG → Stages

Spark divides the DAG into stages.

- **Narrow transformations** (e.g., `filter`) stay in the same stage.
 - **Wide transformations** (e.g., `groupBy`, `join`) require a shuffle and start a new stage.
-

Step 4: Stages → Tasks

Each stage is split into tasks based on the number of data partitions.

Example:

```
200 Partitions
  |
  ▼
200 Tasks
```

Step 5: Cluster Manager

The Driver requests resources from the Cluster Manager.

The Cluster Manager launches executors on worker nodes.

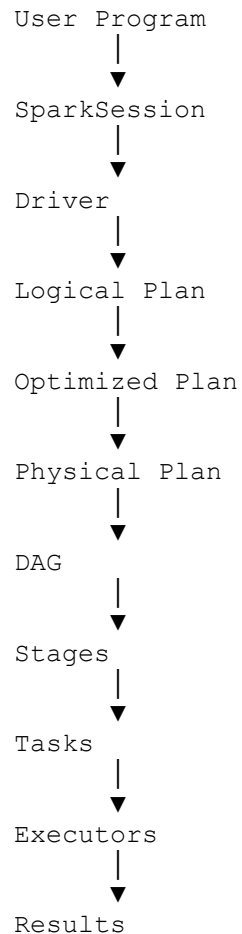
Step 6: Executors

Executors receive tasks, process the data in parallel, and return the results or intermediate data as needed.

Step 7: Driver Receives Results

The Driver collects the final output and returns it to the application.

Execution Flow



Real-World Example

Imagine a company has **1 TB of sales data** stored in HDFS.

Instead of one machine processing all the data:

- The file is divided into many partitions.
- Spark creates one task for each partition.
- Tasks are distributed across executors running on multiple worker nodes.
- Executors process partitions simultaneously.
- The Driver combines the results into the final output.

This parallel processing is one of the main reasons Spark is much faster than processing the data on a single machine.

Component Summary

Component	Responsibility
Driver	Creates execution plan, schedules jobs, monitors execution, collects results
Cluster Manager	Allocates cluster resources and launches executors
Worker Node	Provides CPU and memory resources and hosts executors
Executor	Executes tasks, caches data, and reports status to the Driver
Task	Smallest unit of work; processes one data partition

Interview Questions

Q1. What are the main components of Spark architecture?

- Driver
 - Cluster Manager
 - Worker Nodes
 - Executors
 - Tasks
-

Q2. Why is the Driver called the brain of Spark?

Because it creates the execution plan, optimizes queries, builds the DAG, divides work into stages and tasks, schedules execution, and monitors the entire application.

Q3. What is the difference between a Worker Node and an Executor?

- A **Worker Node** is a machine that provides CPU and memory resources.
- An **Executor** is a JVM process running on a worker node that executes tasks for a specific Spark application.

A worker node can host multiple executors if resources allow.

Q4. What is the smallest unit of execution in Spark?

A **Task**. Each task processes one data partition.

Q5. What is the role of the Cluster Manager?

The Cluster Manager allocates resources, launches executors, and manages CPU and memory across the cluster. It does not execute tasks itself.

Interview Summary

- Apache Spark uses a **master-worker architecture** for distributed processing.
- The **Driver** is the brain of the application. It creates the execution plan, builds the DAG, splits it into stages and tasks, schedules execution, and collects results.
- The **Cluster Manager** allocates resources and launches executors on worker nodes.
- **Worker Nodes** provide the computing resources (CPU and memory) and host executors.
- **Executors** execute tasks, cache data in memory, and communicate progress back to the Driver.
- A **Task** is the smallest unit of work in Spark and processes exactly one data partition.